



# SISTEMAS OPERACIONAIS

## Módulo 02 - Processos

Prof. Daniel Sundfeld  
[daniel.sundfeld@unb.br](mailto:daniel.sundfeld@unb.br)



# MODELO DE PROCESSO

- Todos os computadores modernos são capazes de fazer múltiplas tarefas ao mesmo tempo
- O conceito de processo é um dos mais importantes para qualquer Sistema Operacional
- Um processo é um programa em execução acompanhando de valores de tempo de execução



# MODELO DE PROCESSO

- Processo:
  - Código executável
  - Pilha de execução: contendo valores de variáveis locais
  - Apontador para pilha: registrador da CPU que armazena em qual área de memória está a pilha
  - Contador de programa: registrador da CPU que armazena a próxima instrução a ser executada
  - Valores dos registradores gerais da máquina



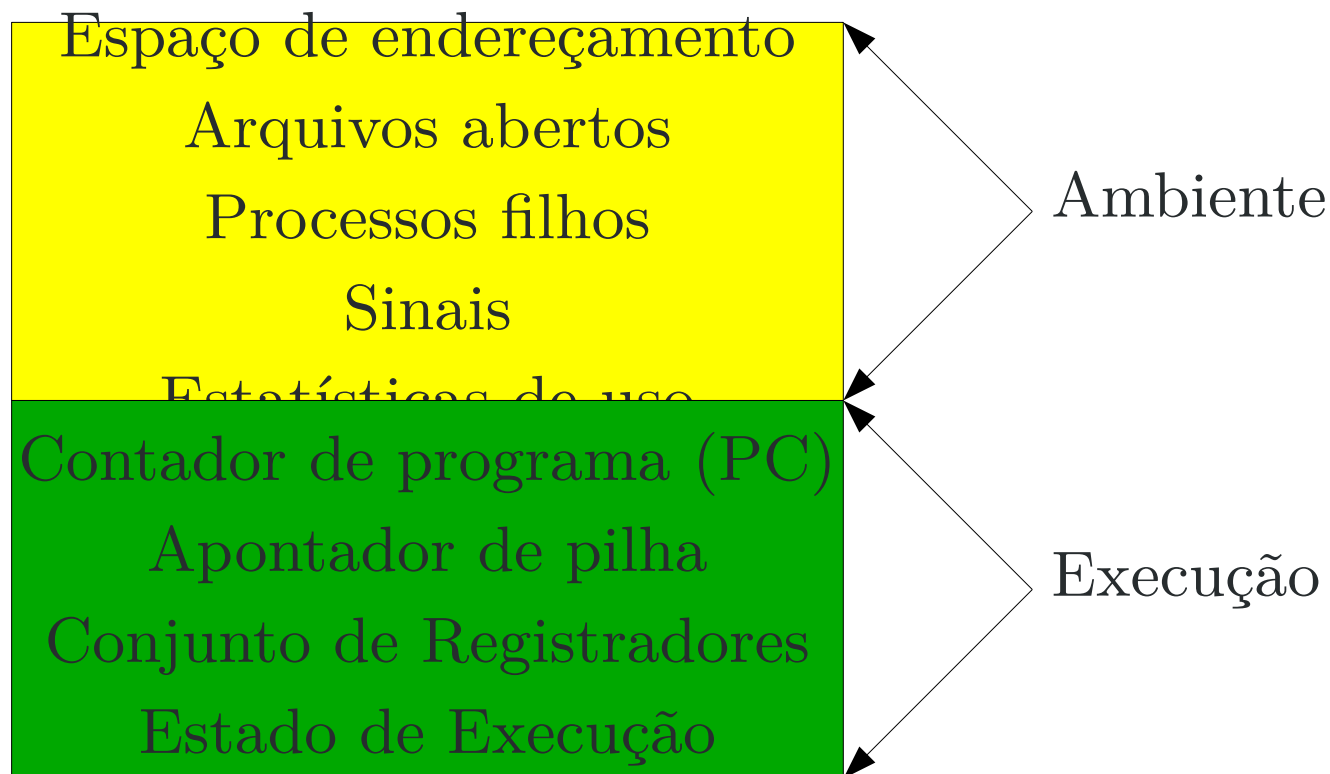
# MODELO DE PROCESSO

- Diferença entre um processo e um programa:
  - Programa: receita de bolo (passivo)
  - Processo: ato de fazer o bolo (ativo)
  - Cozinheiro: CPU
  - Ingredientes do bolo: dados de entrada
  - Bolo: dados de saída
- Processos envolvem uma atividade
- Programas envolvem algoritmos



# MODELO DE PROCESSO

- Informações relevantes a respeito de um processo:





# MODELO DE PROCESSO

- Machado e Maia, dividem o processo em 3 partes: Espaço de Endereçamento, Contexto de Software e Contexto de Hardware
- Execução: Contexto de Hardware
- Ambiente: Espaço de Endereçamento + Contexto de Software



# MODELO DE PROCESSO

- Classificação dos modelos de processos quanto ao custo de troca de contexto e de manutenção
  - Heavyweight (processo tradicional)
  - Lightweight (threads)

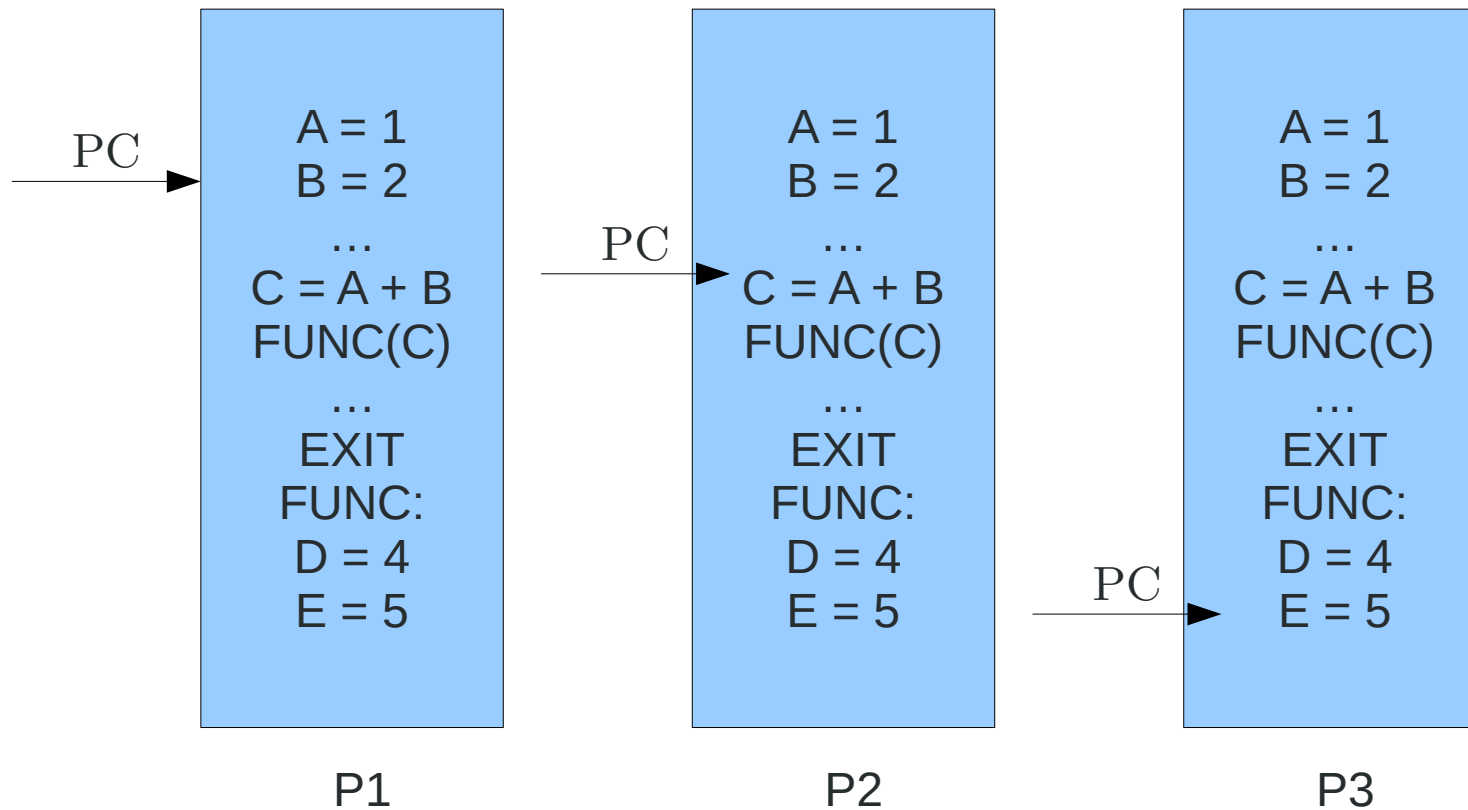


# MODELO DE PROCESSO TRADICIONAL

- Heavyweight
- O processo é composto tanto pelo ambiente como pela execução
- Cada processo possui um único fluxo de controle (contador de programa) e roda de forma independente dos demais
- Em um determinado instante, há vários processos ativos ao mesmo tempo, o processador é chaveado entre diversos processos



# MODELO DE PROCESSO TRADICIONAL





# CRIAÇÃO DE PROCESSOS

- Sistemas de um único propósito possuem um esquema mais simples e previsível de processos em execução
- Contudo, sistemas de propósito geral, que englobam os computadores pessoais atuais, não são previsíveis pois os computadores possuem diferentes programas instalados devido:
  - Hardware diferente
  - Necessidade dos usuários



# CRIAÇÃO DE PROCESSOS

- Desta forma são necessários mecanismos para permitir a criação de processos.
- Os processos são criados:
  - Início do sistema
  - Execução de uma chamada ao sistema de criação de processo por um processo em execução
  - Requisição do usuário
  - Início de um job de lote



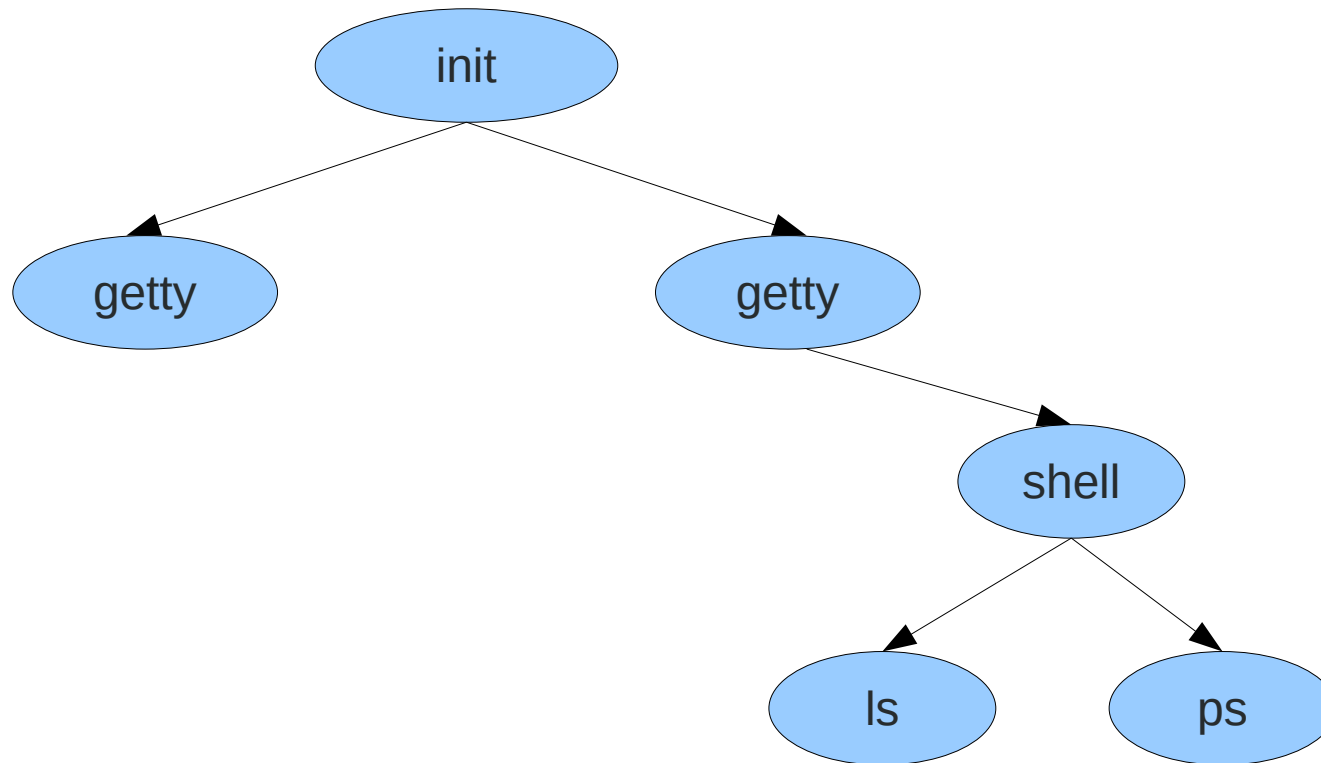
# CRIAÇÃO DE PROCESSOS

- Quando um sistema operacional é carregado, são criados diversos processos
- Alguns executam em primeiro plano e interagem com usuários
- Outros executam em segundo plano: chamados de **daemons**
- Normalmente os processos são criados por outro



# CRIAÇÃO DE PROCESSOS

- Hierarquia em árvore





# CRIAÇÃO DE PROCESSOS

- No Unix há apenas uma forma de criar processos, através de clonagem:
  - `fork()`;
    - Clone idêntico ao processo que chamou
    - Normalmente, executa `execve` depois da chamada para mudar o “programa” em execução
- No Windows, há uma função que trata o processo de criação quanto da carga do programa:
  - `CreateProcess`



# TÉRMINO DE PROCESSOS

- Condições de término de processos:
  - Saída normal (voluntária)
  - Saída por erro (voluntária)
  - Erro fatal (involuntário)
  - Cancelamento por outro processo (involuntário)



# HIERARQUIA DE PROCESSOS

- Quando um processo é criado por outro, eles continuam associados
- Em alguns sistemas, é possível listar o ppid (parent pid)
- Todos os processos são filhos ou descendentes do processo init (atualmente, há uma alternativa: systemd)
- O Windows não apresenta hierarquia de processos



# ESTADOS DE PROCESSOS

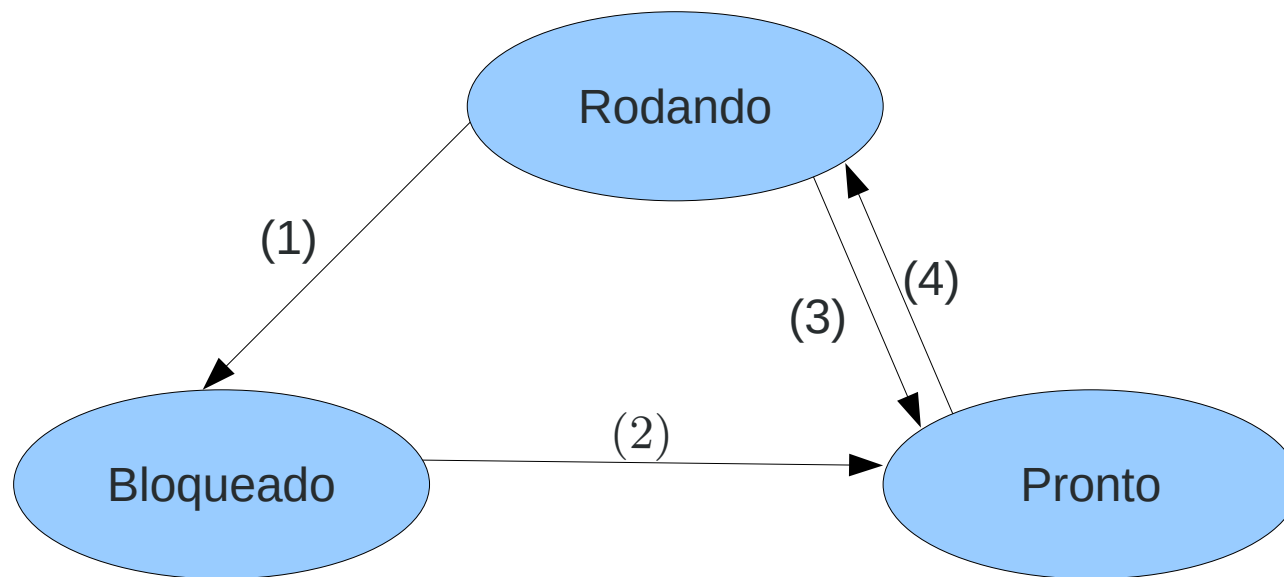
- Apesar de processos serem autossuficientes, muitas vezes necessitam acessar recursos ou comunicar com um processo
- Quando um processo está esperando um evento, dizemos que está bloqueado:
  - Leitura em disco
  - Leitura de rede
  - Esperando entrada do usuário



# ESTADOS DE PROCESSOS

- Os processos podem estar:
  - Rodando
  - Bloqueado
  - Pronto
- Sistemas monoprocessados são aqueles que possuem apenas um único processo rodando

# ESTADOS DE PROCESSOS



(1) O processo bloqueia-se aguardando uma entrada

(2) O evento aguardado pelo processo ocorreu, pode-se iniciar a executar.

(3) O tempo de posse do processador esgotou-se

(4) O processo é escolhido pelo escalonador para executar



# CPU BOUND E I/O BOUND

- Processos podem ser classificados como CPU-bound ou I/O-bound
- Processos CPU-bound (afinidade à CPU) passam a maior parte do tempo usando a CPU, no estado rodando ou no estado pronto
- Processos I/O-bound (afinidade à Entrada e Saída) passam a maior parte do tempo em bloqueado por causarem muitas operações de entrada e saída



# CPU BOUND E I/O BOUND

- Muitas aplicações de computadores pessoais que rodam em primeiro plano são exemplos de aplicações I/O Bound
- Essas aplicações passam muito tempo em estado bloqueado, aguardando as informações dos usuários (digitar um dado de entrada, por exemplo.)



# IMPLEMENTAÇÃO DE PROCESSOS

- Todas as informações sobre um processo são mantidas na tabela de processos
- Visualizar a tabela: “ps aux/CTRL+Alt+Del”
- Contém campos que dizem respeito à gerência do processo, à gerencia da memória e arquivos
- Uma entrada por processo, variam entre os diferentes sistemas operacionais



# TABELA DE PROCESSOS

- Cada processo tem um identificador único conhecido como pid (process id)
- As informações sobre um processo estão na **tabela de processos** (também chamada de **bloco de controle de processo**)
- Durante a execução, o processo compartilha o processador com outros processos em execução



# IMPLEMENTAÇÃO DE PROCESSOS:

## TABELA DE PROCESSOS

| Gerenciamento de processos                                                                                                                                                                                                                                                                                                                                                                                                                                                     | Gerenciamento de memória                                                                                                                                                     | Gerenciamento de arquivos                                                                                                                                                                              |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <ul style="list-style-type: none"><li>• Registradores</li><li>• Contador de programa</li><li>• Palavra de estado do programa</li><li>• Ponteiro de pilha</li><li>• Estado do processo</li><li>• Prioridade</li><li>• Parâmetros de escalonamento</li><li>• Identificador do processo (PID)</li><li>• Processo pai (PPID)</li><li>• Grupo do processo</li><li>• Sinais</li><li>• Momento em que iniciou</li><li>• Tempo de uso de CPU</li><li>• Tempo de CPU do filho</li></ul> | <ul style="list-style-type: none"><li>• Ponteiro para o segmento de código</li><li>• Ponteiro para o segmento de dados</li><li>• Ponteiro para o segmento de pilha</li></ul> | <ul style="list-style-type: none"><li>• Diretório-raiz</li><li>• Diretório de trabalho</li><li>• Descritores de arquivos</li><li>• Identificador do usuário</li><li>• Identificador do grupo</li></ul> |



# IMPLEMENTAÇÃO DE PROCESSOS

- **Troca de contexto:** operação de salvamento dos registradores de um processo e posterior restauração de registradores de outro processo.
- A troca de contexto permite a troca de processador entre os processos
- É a operação básica da multiprogramação



# IMPLEMENTAÇÃO DE PROCESSOS

- Como manter a ilusão de múltiplos processos sequenciais em uma máquina com uma CPU e muitos dispositivos de E/S
- Cada dispositivo de E/S possui possui uma área de memória chamada **vetor de interrupções**
- Esse vetor contém os endereços de procedimentos dos serviços de interrupção



# IMPLEMENTAÇÃO DE PROCESSOS

- O processo P1 solicita a operação
- O SO altera o estado de P1 para bloqueado
- O SO escolhe um dos processos prontos (P2) e o coloca para executar
- A solicitação de P1 é atendida pelo hardware
- O hardware interrompe P2, salvando seu estado de execução na pilha



# IMPLEMENTAÇÃO DE PROCESSOS

- O hardware acessa um endereço de memória física específico que contém o vetor de interrupções
- O vetor de interrupções contém funções de tratamento geradas por cada tipo de dispositivo



# IMPLEMENTAÇÃO DE PROCESSOS

- A rotina de tratamento de interrupção é executada pelo SO
- Os registradores que foram empilhados pelo hardware são salvos na tabela de processo do P2
- A interrupção é tratada
- O processo P1 é colocado na fila de prontos
- O SO acessa a entrada da tabela de processos escolhido e carrega o conteúdo da tabela nos registradores de máquina (restauração)
- O processo escolhido reinicia a execução

# IMPLEMENTAÇÃO DE PROCESSOS

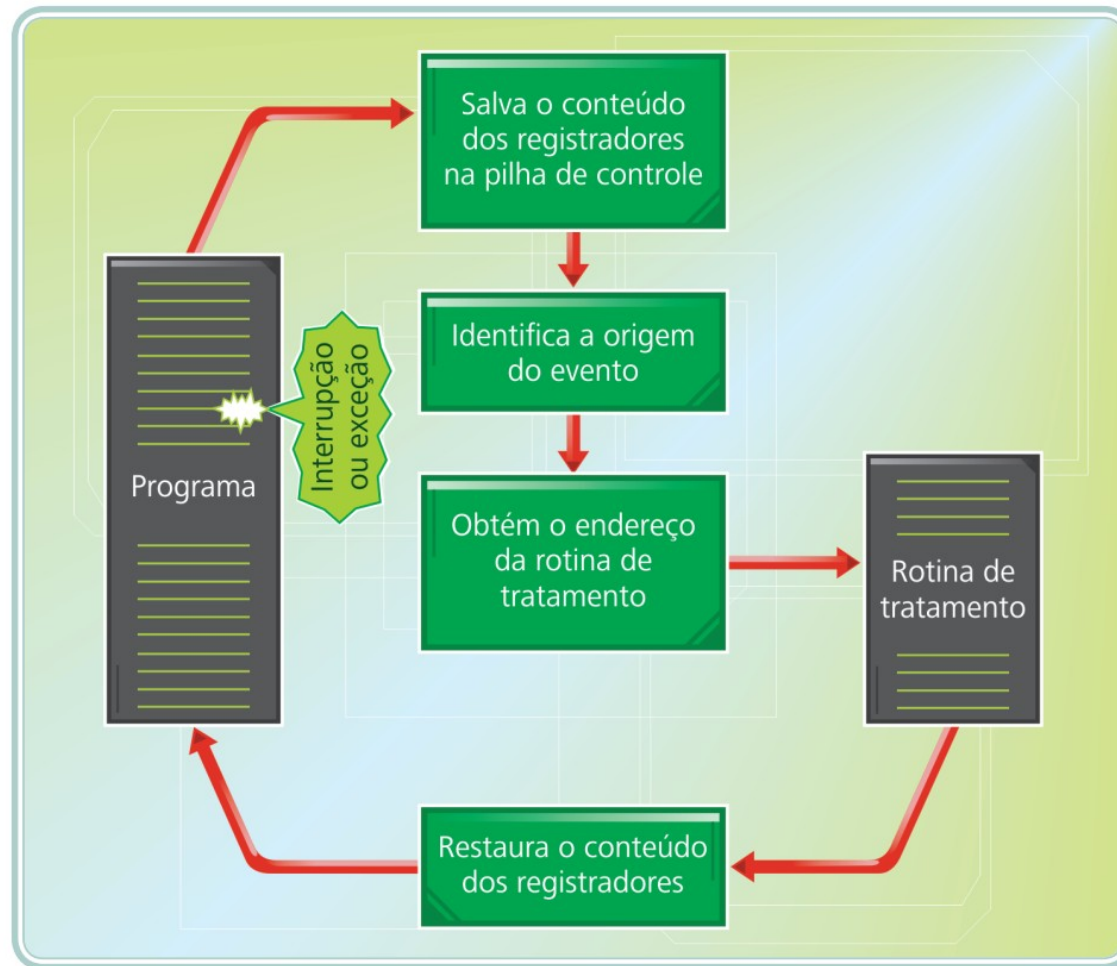


Figura 3.2: Mecanismo de interrupção e exceção



# ESCALONAMENTO DE PROCESSOS

- O Sistema Operacional é responsável por gerenciar o(s) recurso(s) de processamento de um computador
- Essa atividade é conhecida como **Escalonamento de Processador**
- A interação entre processos é realizada através de mecanismos de comunicação



# ESCALONAMENTO DE PROCESSOS

- Quando múltiplos processos encontram-se no estado pronto, é necessário eleger um processo para ser executado
- A parte do SO que faz essa eleição é chamada de **escalonador**
- O algoritmo utilizado chama-se **algoritmo de escalonamento**



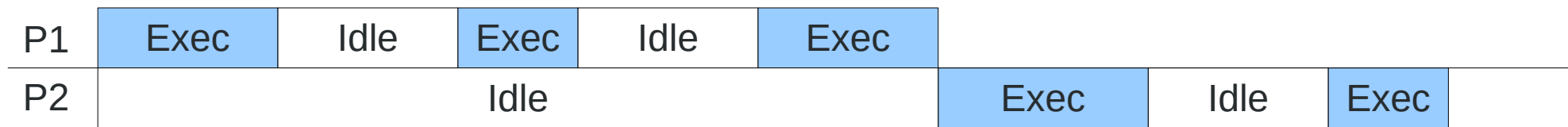
# ESCALONAMENTO DE PROCESSOS

- O algoritmo de escalonamento é o responsável pela determinação de que processo vai rodar e por quanto tempo. Ele define a política de utilização do processador pelos processos
- Quando um processo solicita operações blocantes (E/S), sua execução fica suspensa até que o evento solicitado ocorra

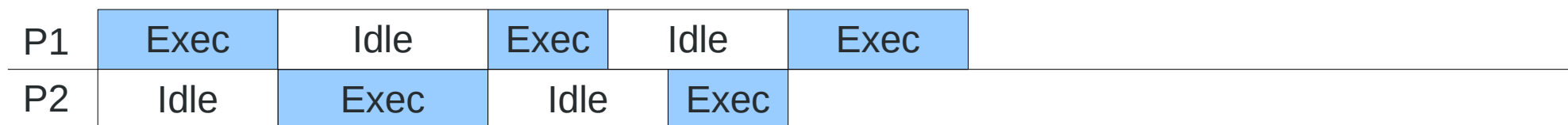


# ESCALONAMENTO DE PROCESSOS

- 2 processos sem concorrência



- 2 processos com concorrência



- Executando concorrentemente pode-se obter uma melhor utilização da CPU, mesmo com apenas uma CPU.  
Especialmente verdade em I/O Bound



# CLASSIFICAÇÃO DOS ESCALONADORES

- Os Escalonadores podem ser classificados em **Preemptivos** e **Não-Preemptivos**
- A preempção é a suspensão temporária da execução de um processo



# ESCALONAMENTO NÃO-PREEMPTIVO

- Escalonador não-preemptivo: Quando um processo obtém o processador, ele roda até o fim ou até que ele peça uma operação que ocasione o seu bloqueio
- Nenhuma entidade externa “tira a CPU à força” do processo



# ESCALONAMENTO PREEMPTIVO

- Cada processo possui um tempo (time-slice) de posse do processador
- Quando este tempo se esgota, o SO retira o processador deste processo e permite que outro processo se execute
- O controle de tempo de execução é feito por interrupção



# ESCALONAMENTO PREEMPTIVO

- Os processadores modernos possuem um clock que gera interrupções a uma frequência determinada
- O SO mantém um contador que é decrementado a cada clock tick
- Se o contador chegar a 0, o tempo de permanência do processo acabou
- O valor inicial deste contador corresponde ao tempo máximo de permanência do processo com a CPU



# ESCALONAMENTO

- Os escalonadores não-preemptivos são de projeto simples, porém permite o abuso no tempo de CPU de um determinado programa
- Neste caso, um processo pode obter o monopólio do processador, impedindo outros processos de rodarem
- Isso viola vários critérios de um bom escalonador (justiça, tempo mínimo de resposta, etc)



# ESCALONAMENTO

- Os escalonadores preemptivos asseguram um uso mais balanceado da CPU e são utilizados na maioria dos SO modernos
- Porém, o projeto de tais escalonadores, além de ser complexo introduz complicações na programação de processos concorrentes
- Os processos podem ser interrompidos em um tempo arbitrário, eles devem proteger suas estruturas de dados contra a interferência de outros processos (regiões críticas)



# CRITÉRIOS DE ESCALONAMENTO

- Ao se projetar um escalonador, devemos observar vários critérios que devem estar presentes em um bom algoritmo de escalonamento:
- **Justiça:** garantir que todos os processos terão chances **justas** de uso de processador. Não são chances iguais
- **Eficiência:** quando existir trabalho a fazer, o processador deve estar ocupado



# CRITÉRIOS DE ESCALONAMENTO

- **Minimizar o tempo de resposta:** reduzindo o tempo dos usuários interativos, reduz o tempo entre a entrada de usuário e a resposta dada (não considera tempo total de execução)
- **Minimizar o turnaround:** reduzir o tempo desde o lançamento do processo até seu término. Soma de: tempo de espera por recursos (memória, processador, E/S) e tempo de utilização da CPU. Mais utilizado em processamento batch



# CRITÉRIOS DE ESCALONAMENTO

- **Minimizar waiting time:** Esse critério visa minimizar o tempo de espera pela CPU
- **Maximizar throughput:** Maximizar o número de tarefas executados em uma unidade de tempo



# CRITÉRIOS DE ESCALONAMENTO

- Não é possível atingir todos esses critérios de escalonamento, muito são contraditórios entre si
- Minimizar turnaround e minimizar tempo de resposta: para que usuários interativos obtenham um tempo de resposta pequeno, geralmente usuários em batch são penalizados com um tempo de resposta maior
- Um algoritmo que maximiza o throughput não é justo com processos de execução demorada



# ALGORITMOS DE ESCALONAMENTO

- O problema a ser resolvido pelos algoritmos de escalonamento: dado um conjunto de processos que devem ser executados, como dividir a utilização do processador entre estes processos?
- Estudar algoritmos clássicos de escalonamento:
  - First Come First Served
  - Round-Robin
  - Prioridades
  - Shortest Job First



# FIRST COME FIRST SERVED (FCFS)

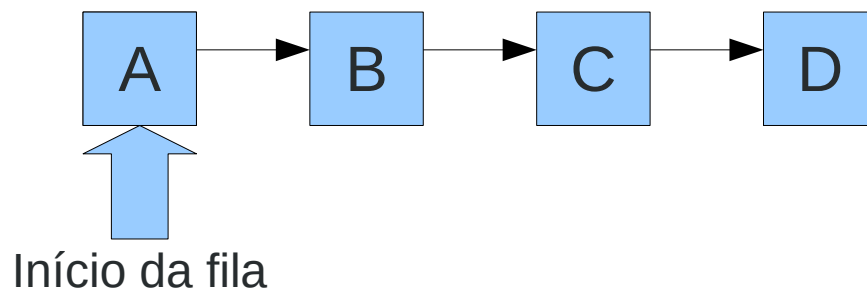
- O processo obtém a CPU de acordo com a ordem da chegada das solicitações. O processo que pede a CPU primeiro obtém a CPU em primeiro lugar
- O escalonamento FCFS é não-preemptivo. Assim, um processo CPU/bound pode fazer com que vários processos esperem por um tempo indeterminado



# FIRST COME FIRST SERVED

- Implementação: processos que solicitam a CPU são colocados em uma fila de prontos, gerenciada segunda a política FIFO (First-In First-Out)

Em execução:

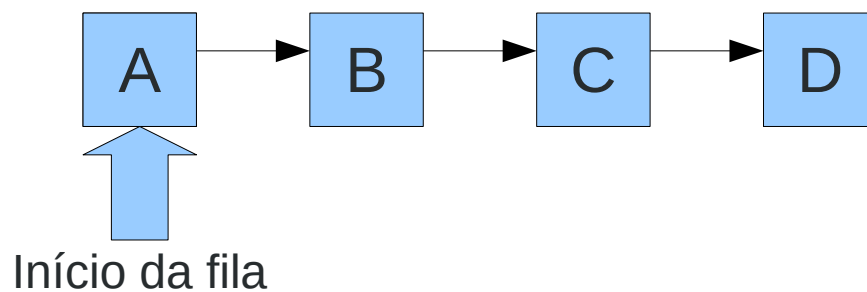


Tempo: 0



# FIRST COME FIRST SERVED

- Primeiro processo da fila é selecionado



Em execução:

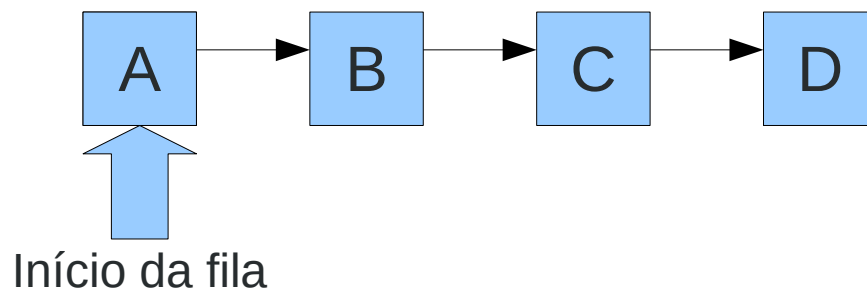


Tempo: 1



# FIRST COME FIRST SERVED

- O processo continua em execução



Em execução:

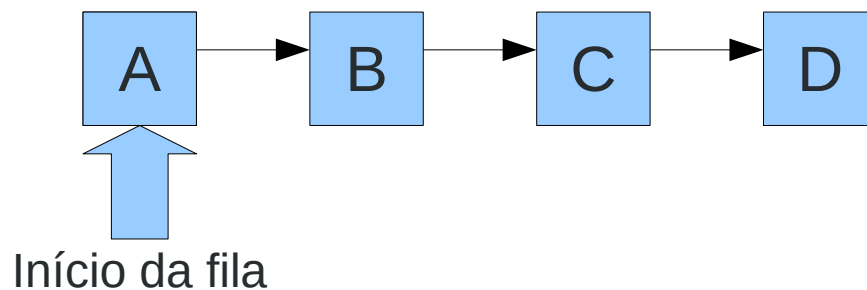


Tempo: 2



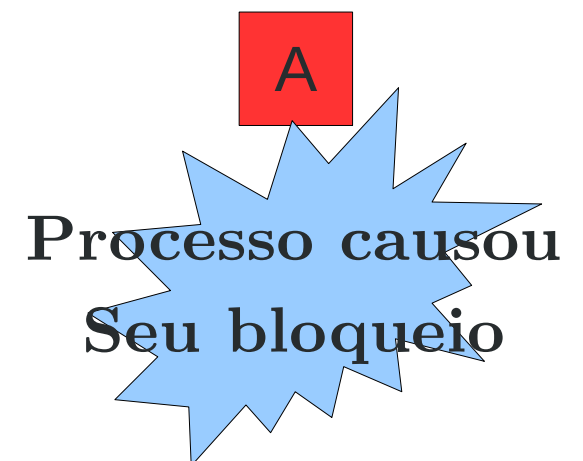
# FIRST COME FIRST SERVED

- O processo continua em execução até causar seu bloqueio



Tempo: 3

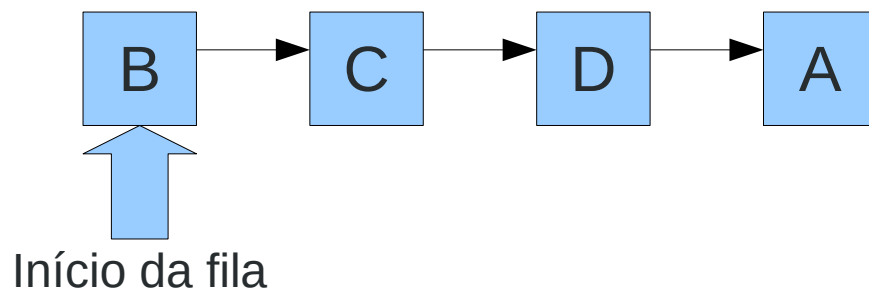
Em execução:





# FIRST COME FIRST SERVED

- O próximo processo da fila é selecionado



Em execução:

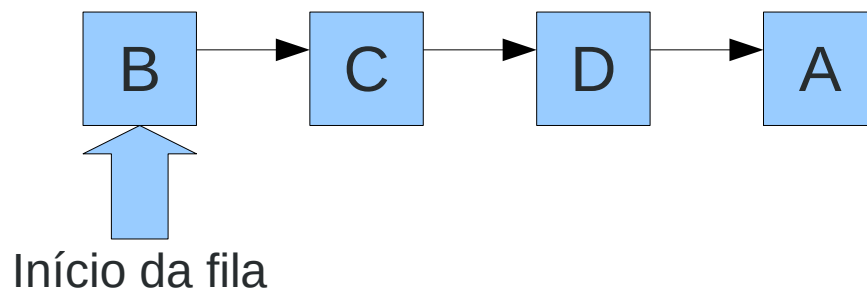


Tempo: 4



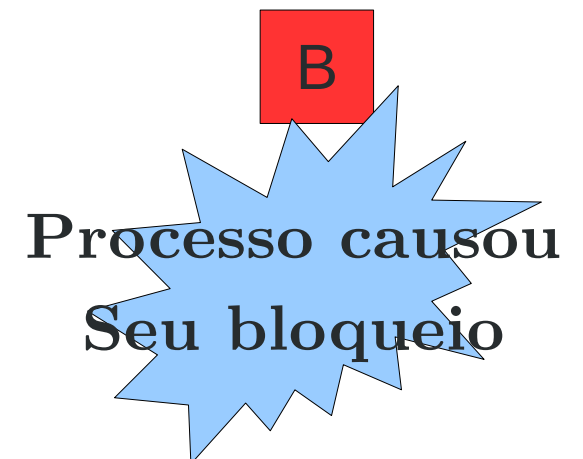
# FIRST COME FIRST SERVED

- O processo B causa seu bloqueio



Tempo: 5

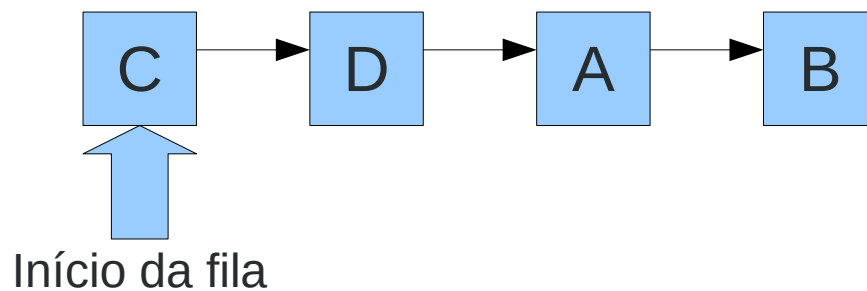
Em execução:





# FIRST COME FIRST SERVED

- O próximo processo é selecionado



Em execução:



Tempo: 6



# FIRST COME FIRST SERVED (FCFS)

- Vantagens:
  - Simples de ser implementado
  - Algoritmo **eficiente**: a CPU sempre é utilizada
- Desvantagens:
  - Impossibilidade de se prever quando um processo vai iniciar
  - Tempo média de espera de processos não é priorizado
  - Processos que usam muito a CPU levam vantagens sobre processos que causam muito seu bloqueio



# FIRST COME FIRST SERVED

- Problema: dados os seguintes processos, tempo de execução e tempo de chegada. Calcule o tempo de serviço e tempo médio de espera
- $\text{Tempo de Espera} = \text{Tempo de Serviço} - \text{Tempo de Chegada}$



# FIRST COME FIRST SERVED

- $\text{Tempo de Espera} = \text{Tempo de Serviço} - \text{Tempo de Chegada}$
- Nenhum dos processo gera chamada blocantes

| Processo | Tempo de Chegada | Tempo de Execução | Tempo de Serviço | Tempo de Espera |
|----------|------------------|-------------------|------------------|-----------------|
| P1       | 0                | 8                 |                  |                 |
| P2       | 1                | 2                 |                  |                 |
| P3       | 2                | 5                 |                  |                 |
| P4       | 3                | 4                 |                  |                 |



# ESCALONAMENTO DE PROCESSOS

| Exec |
|------|
| Idle |

Executando  
Esperando

Tempo: 0

Fila de Processos: P1

Processo em execução:

0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20

P1

---

---

---



# ESCALONAMENTO DE PROCESSOS

Tempo: 1

Fila de Processos: P2

Processo em execução: P1

| Exec |
|------|
| Idle |

Executando  
Esperando

0 1

P1

Exec

2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20

P2



# ESCALONAMENTO DE PROCESSOS

Tempo: 2

Fila de Processos: P2, P3

Processo em execução: P1

|    | 0    | 1    | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 | 12 | 13 | 14 | 15 | 16 | 17 | 18 | 19 | 20 |
|----|------|------|---|---|---|---|---|---|---|---|----|----|----|----|----|----|----|----|----|----|----|
| P1 | Exec |      |   |   |   |   |   |   |   |   |    |    |    |    |    |    |    |    |    |    |    |
| P2 |      | Idle |   |   |   |   |   |   |   |   |    |    |    |    |    |    |    |    |    |    |    |
| P3 |      |      |   |   |   |   |   |   |   |   |    |    |    |    |    |    |    |    |    |    |    |



# ESCALONAMENTO DE PROCESSOS

|      |
|------|
| Exec |
| Idle |

Executando  
Esperando

Tempo: 3

Fila de Processos: P2, P3, P4

Processo em execução: P1

|    | 0    | 1 | 2    | 3    | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 | 12 | 13 | 14 | 15 | 16 | 17 | 18 | 19 | 20 |
|----|------|---|------|------|---|---|---|---|---|---|----|----|----|----|----|----|----|----|----|----|----|
| P1 | Exec |   |      |      |   |   |   |   |   |   |    |    |    |    |    |    |    |    |    |    |    |
| P2 |      |   | Idle |      |   |   |   |   |   |   |    |    |    |    |    |    |    |    |    |    |    |
| P3 |      |   |      | Idle |   |   |   |   |   |   |    |    |    |    |    |    |    |    |    |    |    |
| P4 |      |   |      |      |   |   |   |   |   |   |    |    |    |    |    |    |    |    |    |    |    |



# ESCALONAMENTO DE PROCESSOS

|      |
|------|
| Exec |
| Idle |

Executando  
Esperando

Tempo: 4

Fila de Processos: P2, P3, P4

Processo em execução: P1

|    | 0    | 1 | 2    | 3    | 4    | 5 | 6 | 7 | 8 | 9 | 10 | 11 | 12 | 13 | 14 | 15 | 16 | 17 | 18 | 19 | 20 |
|----|------|---|------|------|------|---|---|---|---|---|----|----|----|----|----|----|----|----|----|----|----|
| P1 | Exec |   |      |      |      |   |   |   |   |   |    |    |    |    |    |    |    |    |    |    |    |
| P2 |      |   | Idle |      |      |   |   |   |   |   |    |    |    |    |    |    |    |    |    |    |    |
| P3 |      |   |      | Idle |      |   |   |   |   |   |    |    |    |    |    |    |    |    |    |    |    |
| P4 |      |   |      |      | Idle |   |   |   |   |   |    |    |    |    |    |    |    |    |    |    |    |



# ESCALONAMENTO DE PROCESSOS

Tempo: 5

Fila de Processos: P2, P3, P4

Processo em execução: P1

|    |                 |   |            |      |          |   |   |   |   |   |    |    |    |    |                                                           |    |    |    |    |    |    |  |  |  |
|----|-----------------|---|------------|------|----------|---|---|---|---|---|----|----|----|----|-----------------------------------------------------------|----|----|----|----|----|----|--|--|--|
|    | <div>Exec</div> |   | Executando |      | Tempo: 5 |   |   |   |   |   |    |    |    |    | Fila de Processos: P2, P3, P4<br>Processo em execução: P1 |    |    |    |    |    |    |  |  |  |
|    | <div>Idle</div> |   | Esperando  |      |          |   |   |   |   |   |    |    |    |    |                                                           |    |    |    |    |    |    |  |  |  |
|    | 0               | 1 | 2          | 3    | 4        | 5 | 6 | 7 | 8 | 9 | 10 | 11 | 12 | 13 | 14                                                        | 15 | 16 | 17 | 18 | 19 | 20 |  |  |  |
| P1 | Exec            |   |            |      |          |   |   |   |   |   |    |    |    |    |                                                           |    |    |    |    |    |    |  |  |  |
| P2 |                 |   | Idle       |      |          |   |   |   |   |   |    |    |    |    |                                                           |    |    |    |    |    |    |  |  |  |
| P3 |                 |   |            | Idle |          |   |   |   |   |   |    |    |    |    |                                                           |    |    |    |    |    |    |  |  |  |
| P4 |                 |   |            |      | Idle     |   |   |   |   |   |    |    |    |    |                                                           |    |    |    |    |    |    |  |  |  |



# ESCALONAMENTO DE PROCESSOS

Tempo: 6

Fila de Processos: P2, P3, P4

Processo em execução: P1

|                                 |      |            |          |      |      |   |   |   |   |   |    |    |                               |    |    |    |    |    |    |    |    |  |
|---------------------------------|------|------------|----------|------|------|---|---|---|---|---|----|----|-------------------------------|----|----|----|----|----|----|----|----|--|
| <div>Exec</div> <div>Idle</div> |      | Executando | Tempo: 6 |      |      |   |   |   |   |   |    |    | Fila de Processos: P2, P3, P4 |    |    |    |    |    |    |    |    |  |
|                                 |      | Esperando  |          |      |      |   |   |   |   |   |    |    | Processo em execução: P1      |    |    |    |    |    |    |    |    |  |
|                                 | 0    | 1          | 2        | 3    | 4    | 5 | 6 | 7 | 8 | 9 | 10 | 11 | 12                            | 13 | 14 | 15 | 16 | 17 | 18 | 19 | 20 |  |
| P1                              | Exec |            |          |      |      |   |   |   |   |   |    |    |                               |    |    |    |    |    |    |    |    |  |
| P2                              |      |            | Idle     |      |      |   |   |   |   |   |    |    |                               |    |    |    |    |    |    |    |    |  |
| P3                              |      |            |          | Idle |      |   |   |   |   |   |    |    |                               |    |    |    |    |    |    |    |    |  |
| P4                              |      |            |          |      | Idle |   |   |   |   |   |    |    |                               |    |    |    |    |    |    |    |    |  |



# ESCALONAMENTO DE PROCESSOS

|      |
|------|
| Exec |
| Idle |

Executando  
Esperando

Tempo: 7

Fila de Processos: P2, P3, P4

Processo em execução: P1

|    | 0    | 1 | 2    | 3    | 4    | 5 | 6 | 7 | 8 | 9 | 10 | 11 | 12 | 13 | 14 | 15 | 16 | 17 | 18 | 19 | 20 |
|----|------|---|------|------|------|---|---|---|---|---|----|----|----|----|----|----|----|----|----|----|----|
| P1 | Exec |   |      |      |      |   |   |   |   |   |    |    |    |    |    |    |    |    |    |    |    |
| P2 |      |   | Idle |      |      |   |   |   |   |   |    |    |    |    |    |    |    |    |    |    |    |
| P3 |      |   |      | Idle |      |   |   |   |   |   |    |    |    |    |    |    |    |    |    |    |    |
| P4 |      |   |      |      | Idle |   |   |   |   |   |    |    |    |    |    |    |    |    |    |    |    |



# ESCALONAMENTO DE PROCESSOS

| Exec |
|------|
| Idle |

Executando  
Esperando

Tempo: 8

Fila de Processos: P2, P3, P4

Processo em execução: P1

|    | 0    | 1 | 2    | 3    | 4    | 5 | 6 | 7 | 8 | 9 | 10 | 11 | 12 | 13 | 14 | 15 | 16 | 17 | 18 | 19 | 20 |
|----|------|---|------|------|------|---|---|---|---|---|----|----|----|----|----|----|----|----|----|----|----|
| P1 | Exec |   |      |      |      |   |   |   |   |   |    |    |    |    |    |    |    |    |    |    |    |
| P2 |      |   | Idle |      |      |   |   |   |   |   |    |    |    |    |    |    |    |    |    |    |    |
| P3 |      |   |      | Idle |      |   |   |   |   |   |    |    |    |    |    |    |    |    |    |    |    |
| P4 |      |   |      |      | Idle |   |   |   |   |   |    |    |    |    |    |    |    |    |    |    |    |

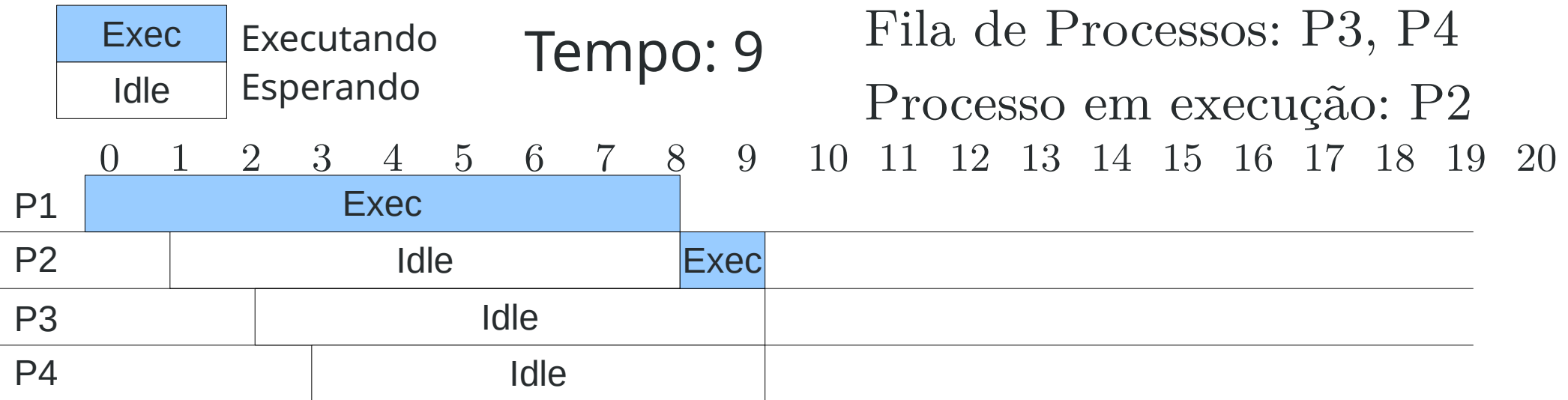


# ESCALONAMENTO DE PROCESSOS

Tempo: 9

Fila de Processos: P3, P4

Processo em execução: P2





# ESCALONAMENTO DE PROCESSOS

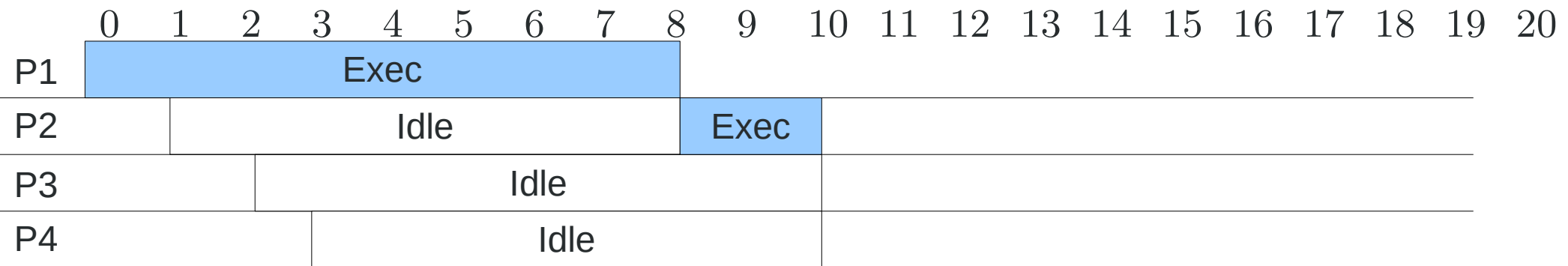
Tempo: 10

Fila de Processos: P3, P4

Processo em execução: P2

| Exec |
|------|
| Idle |

Executando  
Esperando



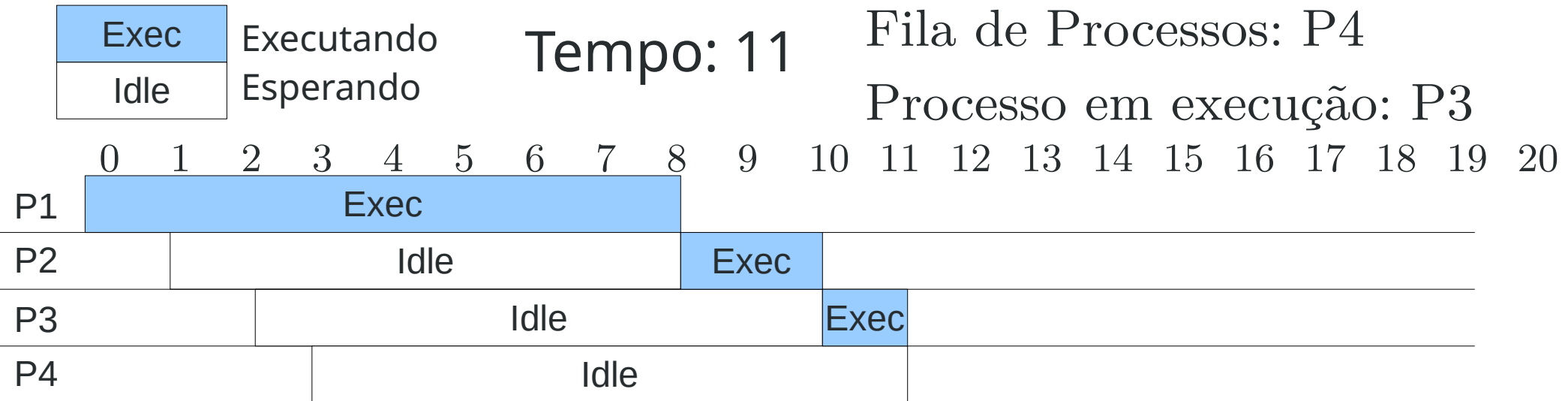


# ESCALONAMENTO DE PROCESSOS

Tempo: 11

Fila de Processos: P4

Processo em execução: P3





# ESCALONAMENTO DE PROCESSOS

Tempo: 12

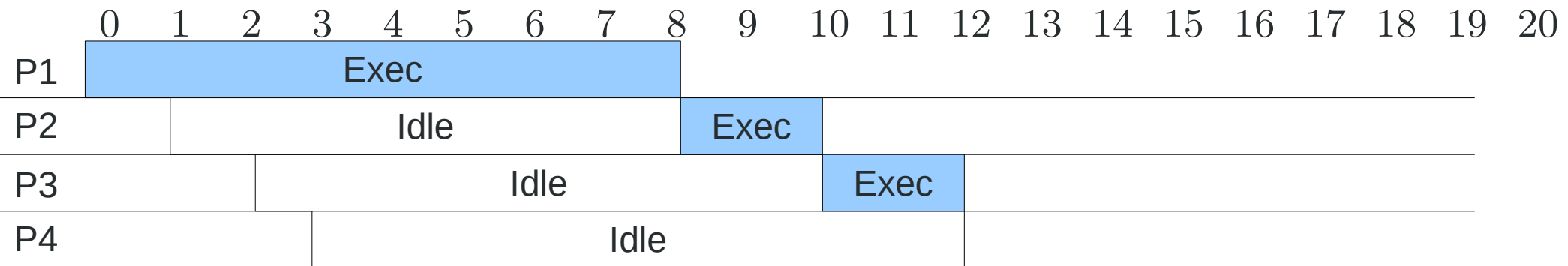
Fila de Processos: P4

Processo em execução: P3

|      |
|------|
| Exec |
| Idle |

Executando

Esperando



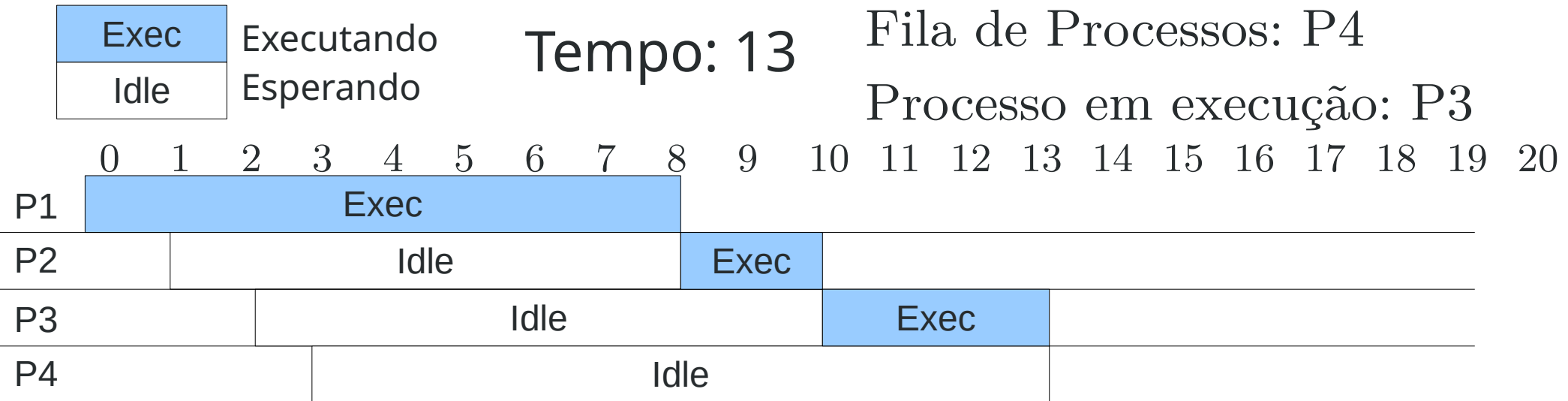


# ESCALONAMENTO DE PROCESSOS

Tempo: 13

Fila de Processos: P4

Processo em execução: P3



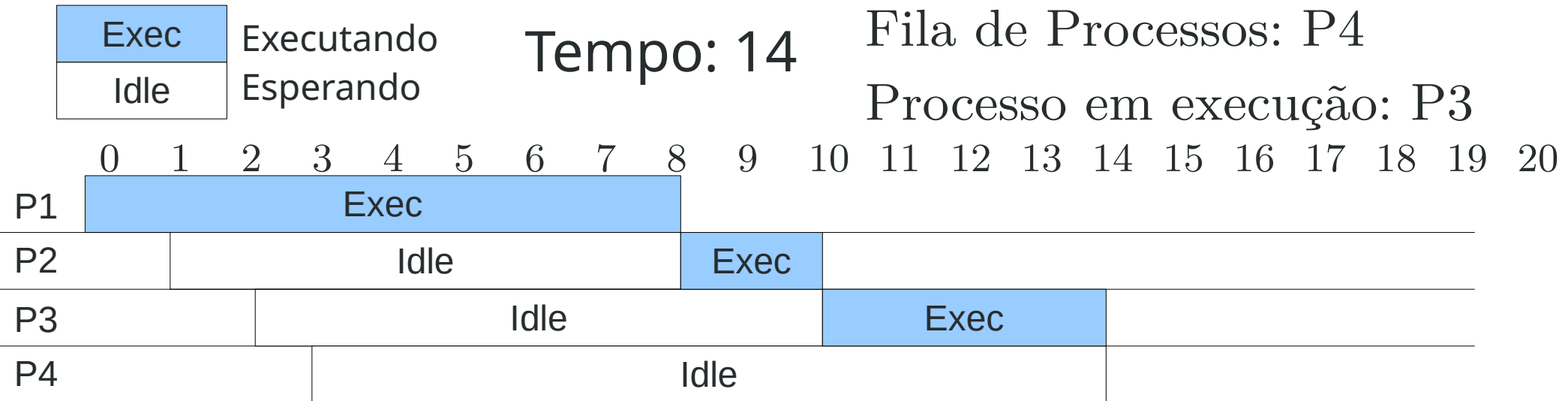


# ESCALONAMENTO DE PROCESSOS

Tempo: 14

Fila de Processos: P4

Processo em execução: P3



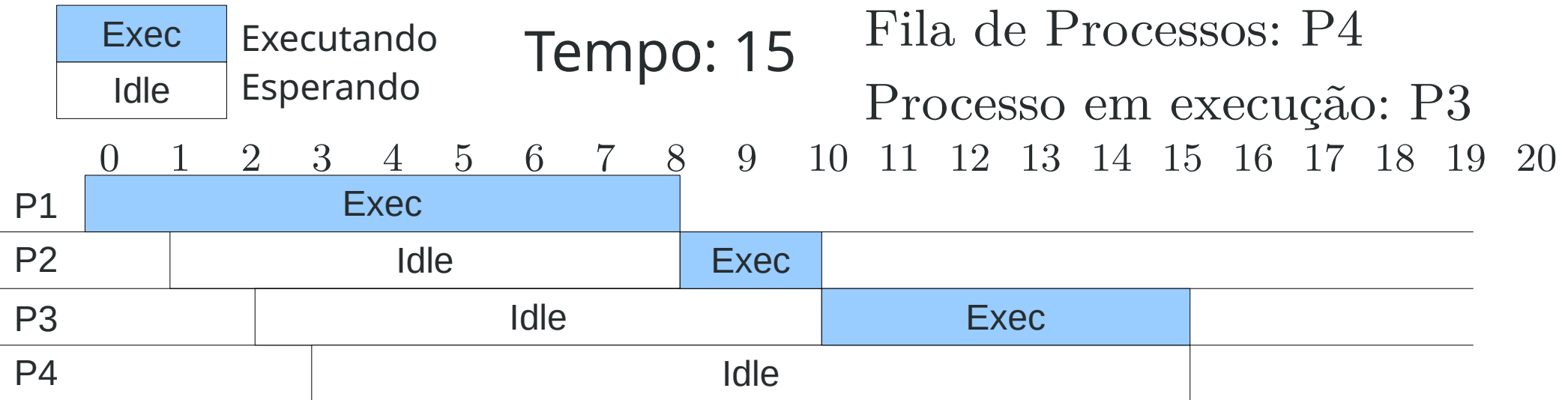


# ESCALONAMENTO DE PROCESSOS

Tempo: 15

Fila de Processos: P4

Processo em execução: P3



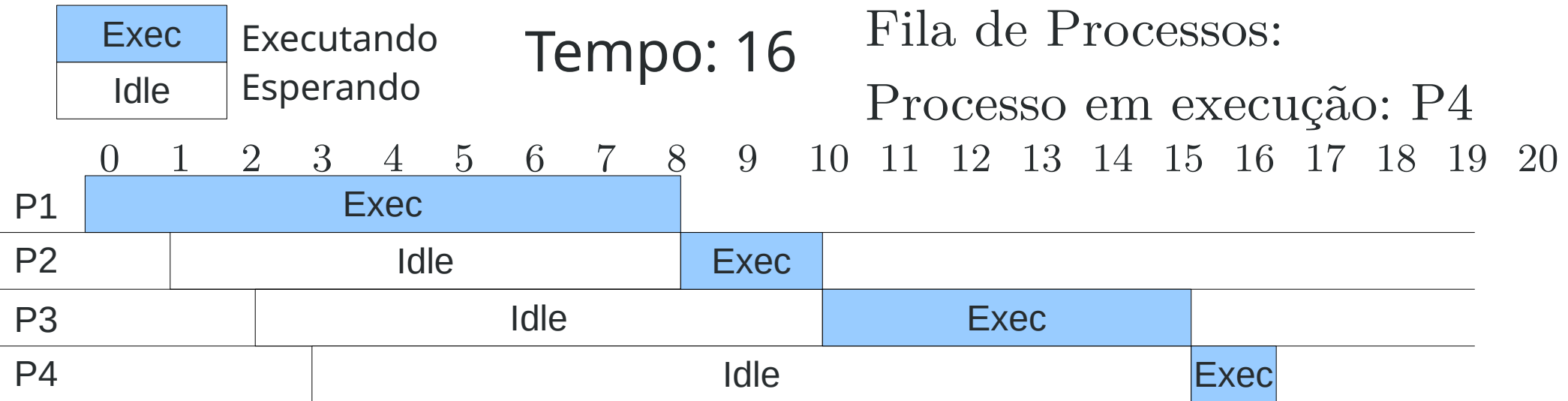


# ESCALONAMENTO DE PROCESSOS

Tempo: 16

Fila de Processos:

Processo em execução: P4



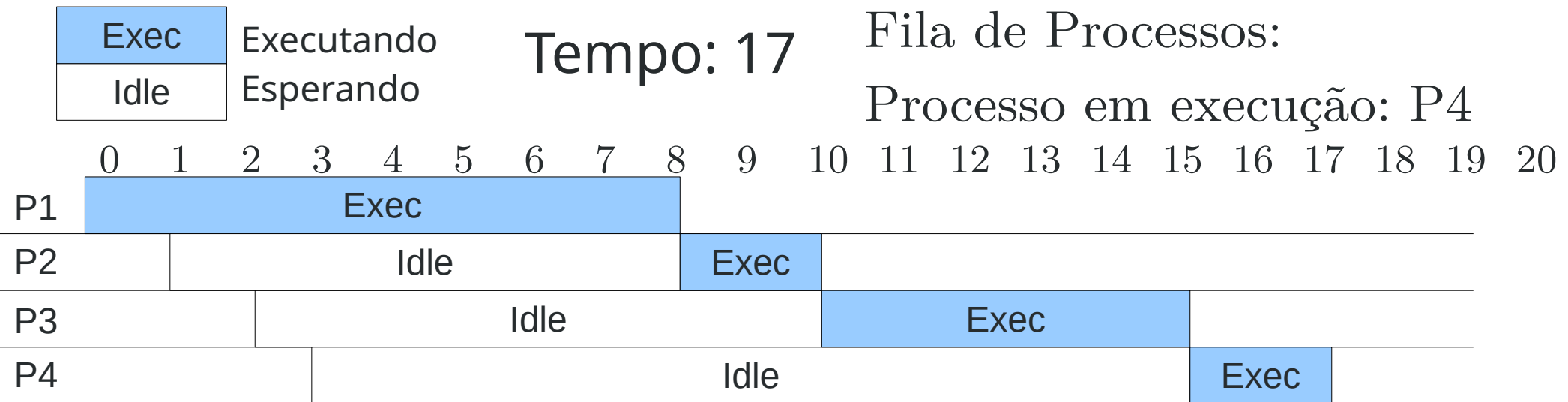


# ESCALONAMENTO DE PROCESSOS

Tempo: 17

Fila de Processos:

Processo em execução: P4



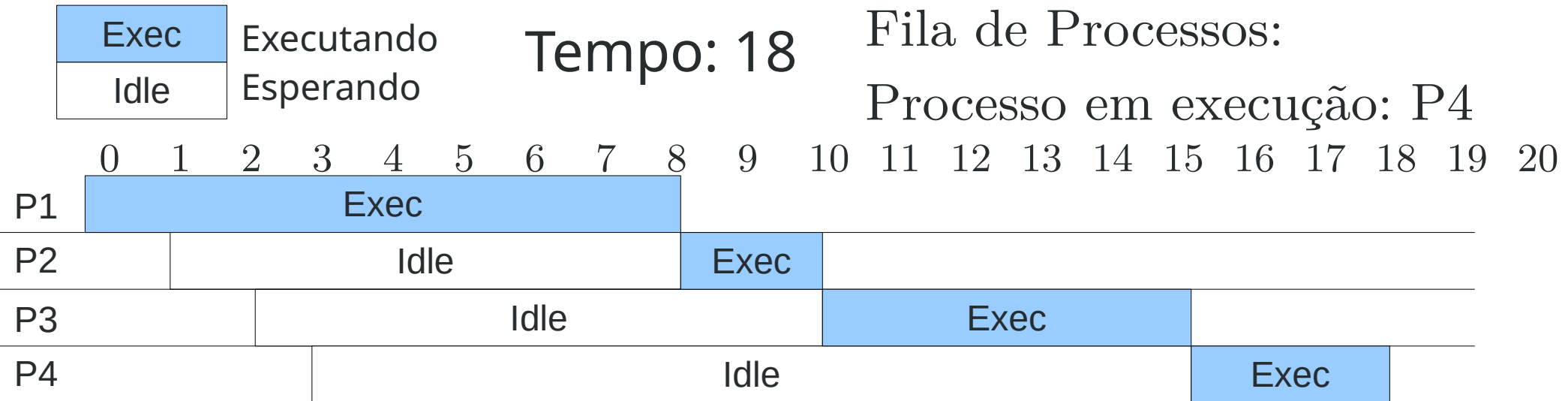


# ESCALONAMENTO DE PROCESSOS

Tempo: 18

Fila de Processos:

Processo em execução: P4



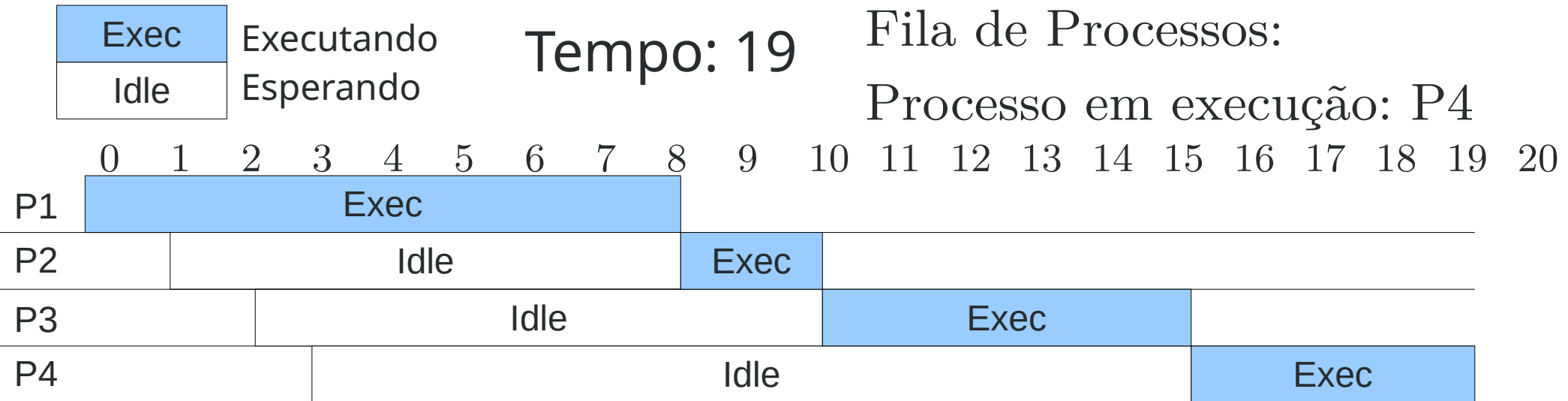


# ESCALONAMENTO DE PROCESSOS

Tempo: 19

Fila de Processos:

Processo em execução: P4



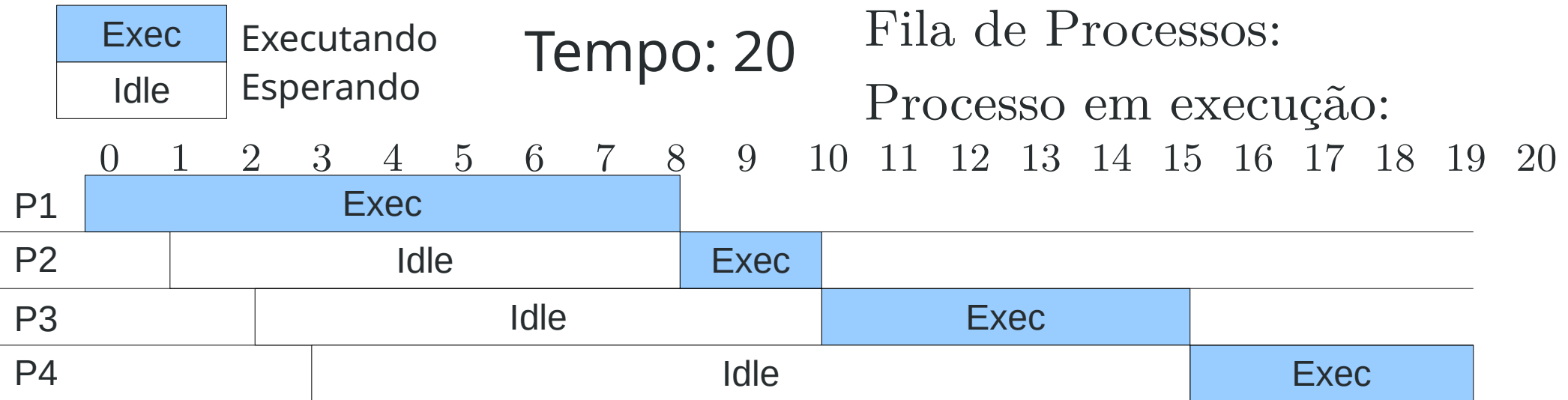


# ESCALONAMENTO DE PROCESSOS

Tempo: 20

Fila de Processos:

Processo em execução:





# FIRST COME FIRST SERVED

- $\text{Tempo de Espera} = \text{Tempo de Serviço} - \text{Tempo de Chegada}$
- Nenhum dos processo gera chamada blocantes

| Processo | Tempo de Chegada | Tempo de Execução | Tempo de Serviço | Tempo de Espera |
|----------|------------------|-------------------|------------------|-----------------|
| P1       | 0                | 8                 | 0                | 0               |
| P2       | 1                | 2                 | 8                | 7               |
| P3       | 2                | 5                 | 10               | 8               |
| P4       | 3                | 4                 | 15               | 12              |



# FIRST COME FIRST SERVED

- **Solução:** Tempo médio de espera  
 $(0 + 7 + 8 + 12) / 4 = 6,75$



# FIRST COME FIRST SERVED

- **Justiça:** Não. Um processo pode monopolizar
- **Eficiência:** OK. A CPU sempre é utilizada, se existe trabalho
- **Minimizar o tempo de resposta:** Não. Caso um processo monopolize
- **Minimizar o turnaround:** Tarefas longas sendo executadas primeiro, aumenta turnaround (Ver exemplo do SJF a frente)
- **Minimizar waiting time:** O tempo de espera não é considerado durante o escalonamento
- **Maximizar throughput:** Se a primeira tarefa for longa, o tempo de término fica alta



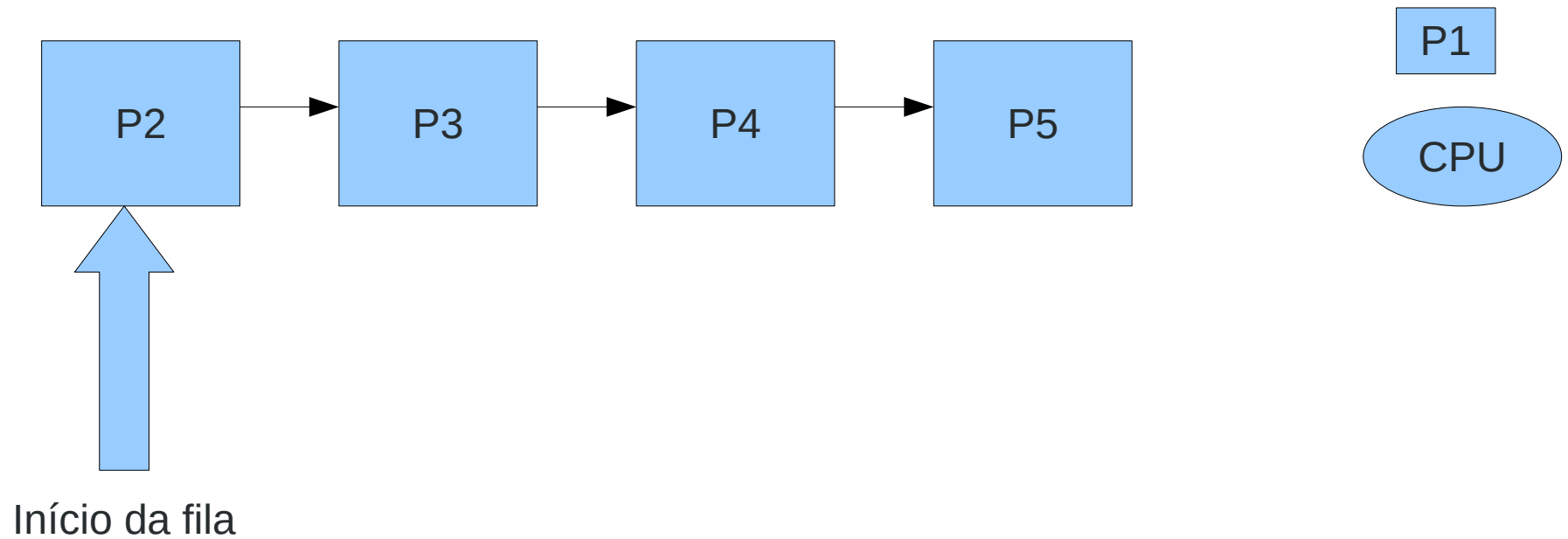
# ROUND-ROBIN

- Cada processo tem o direito de usar o processador por um intervalo de tempo pré-definido. Este intervalo de tempo é denominado quantum
- Quando o quantum se esgota, o processador é dado a outro processo
- Algoritmo por alternância circular
- Algoritmo justo



# ROUND-ROBIN

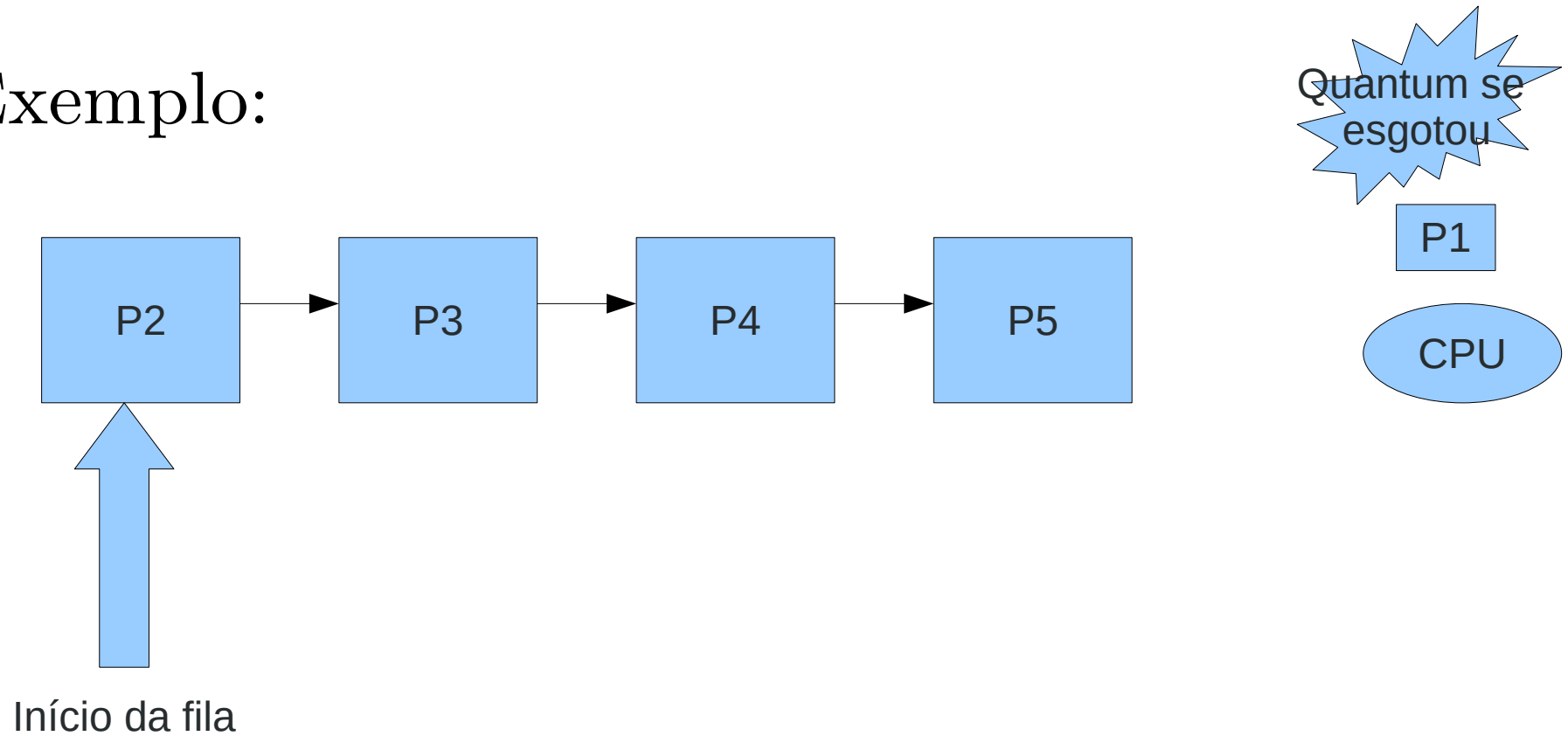
- Exemplo:





# ROUND-ROBIN

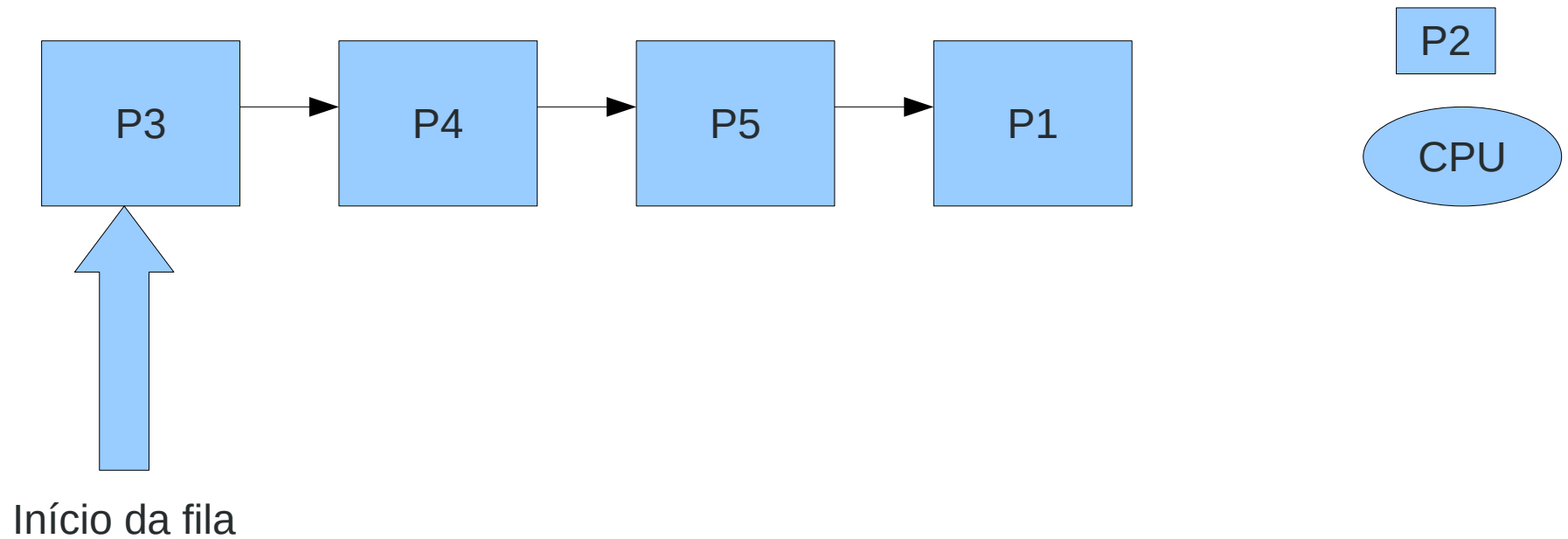
- Exemplo:





# ROUND-ROBIN

- Exemplo:





# ROUND-ROBIN

- Um dos maiores problemas do algoritmo de escalonamento round-robin diz a respeito da determinação de um bom valor a ser atribuído ao quantum
- Para determinar esse valor, deve-se considerar o tempo médio da troca de contexto e o tempo de resposta desejado
  - Qual a influência que o tempo da troca de contexto?
- Quantum padrão 100 ms



# ROUND-ROBIN

- Quantum muito longo: FCFS
- Quantum muito curto: perda de vários requisitos, tais como eficiência
- Para o algoritmo ser eficiente é necessário que o tempo de troca de contexto seja consideravelmente menor que a execução do programa
  - Verdade para os sistemas computacionais modernos



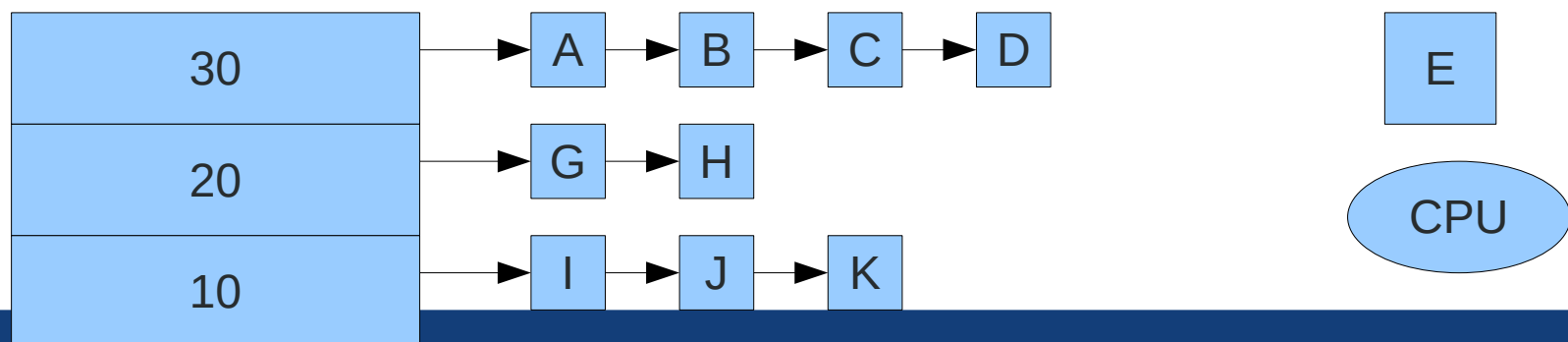
# ESCALONAMENTO COM PRIORIDADES

- Baseia-se no fato que alguns processos são prioritários e devem ser executados antes dos outros
- Sistemas Operacionais em Tempo Real
- Cada processo é atribuída uma prioridade.  
Processos com prioridade maior rodam primeiro



# ESCALONAMENTO COM PRIORIDADES

- As prioridades podem ser atribuídas de duas formas: estática ou dinâmica
- **Estática:** os processos são divididos em classes e a cada classe é atribuída uma prioridade. A cada prioridade existe uma fila de prontos associada





# ESCALONAMENTO COM PRIORIDADES

- **Dinâmica:** o sistema analisa o comportamento dos processos e atribui prioridades favorecendo um certo tipo de comportamento
  - Processos I/O devem possuir prioridade alta
  - Prioridade dinâmica:  $1/f$ , onde  $f$  é a fração do quantum de tempo usada na última rodada do processo



# SHORTEST JOB FIRST

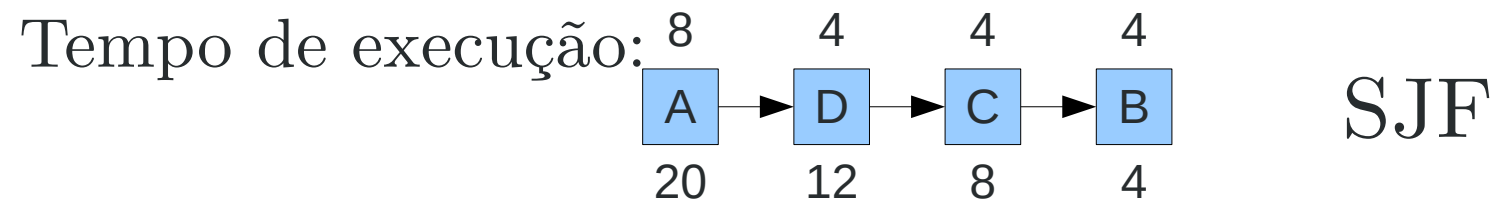
- Algoritmo projetado para sistemas em lote
- Reduzir o tempo de turnaround (tempo de lançamento do processo até seu término)
- Requer que o tempo total de execução do processo seja conhecido antes do seu início
- Algoritmo: Dado um conjunto de processos, execute os de menor tempo de execução antes



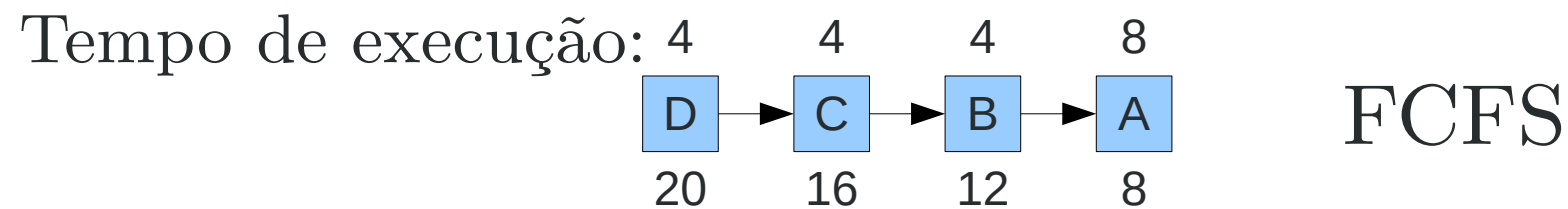
# SHORTEST JOB FIRST

Início

- Exemplo:



Turnaround:



Turnaround:



# SHORTEST JOB FIRST

- Aplicação para sistemas interativos
- Processo interativo:
  - Espera comando
  - Executa comando
- Se considerarmos cada “executa comando” como um trabalho, podemos aplicar o SJF para processos interativos



# SHORTEST JOB FIRST

- Desvantagem: como determinar o tempo de execução do comando?
- **Problema da parada é indecidível**
- Necessário combinar técnicas para estimar valores baseados em execuções anteriores (aging)
- Quando um valor fica antigo, ele praticamente não influencia na estimativa



# ESCALONAMENTO EM DOIS NÍVEIS

- Um caso típico de escalonamento em dois níveis é o algoritmo que considera tanto os processos que estão em memória como os processos que estão em disco
  - Primeiro nível: manipula os processos que estão carregados em memória (FCFS, SJF, ...)
  - Segundo Nível: examina periodicamente o tempo de execução dos processos e os tira ou carrega em memória
- Quando o modelo de processos inclui threads, podemos ter também em 2 níveis. O primeiro determina qual processo irá rodar e o segundo determina qual thread do processo selecionado irá executar



## Referências

- Capítulo 2 – TANENBAUM, A. S. *Sistemas Operacionais Modernos*. 4<sup>a</sup> ed. Prentice Hall, 2016.
- Capítulo 5 e 8 – MACHADO, F. B; MAIA, L. P. *Arquitetura de Sistemas Operacionais*. 5<sup>a</sup> ed. Rio de Janeiro: LTC, 2013.